DuocUC® INFORMÁTICA Y TELECOMUNICACIONES



INFORME DE EVIDENCIAS – PARCIAL No. 2

Integrantes:

Belén Lisboa

Marco Tassara

Martín Villegas

Sección: TCY0101

Profesor: Christian Rodrigo Alvarez Lobos

Nombre de la Asignatura:

TCY0101 – TÉCNICAS DE CALIDAD DE SOFTWARE

Ejecución de Pruebas Automatizadas

Sistema: Gestor de Inventario

Realizado por: Martín Villegas

EVIDENCIA DE EJECUCIÓN DE PRUEBAS: **AUTOMATIZADAS**		
ID. Prueba: SELENIUM-001	Id. Caso de Uso Asociado: Login, navegacion al menu, acceso a Productos, cierre de sesion	Id. Del Requerimiento Asociado: RF de autenticacion y navegacion del frontend
Fecha de Ejecucion: 20-10-2026	Herramienta(s) Ocupada(s): Selenium WebDriver 4.23 (Java) + Chrome headless	Alcance: frontend Next.js (localhost:3000), flujo completo de usuario real en navegador
Ambiente de Prueba:	Local, Chrome 155 headless, proyecto Maven independiente (selenium-tests/), contra el frontend y backend ya levantados	
Resultado Esperado:	Que un login valido redirija al menu, que un login invalido muestre error y no redirija, que la navegacion a Productos cargue el listado, y que cerrar sesion vuelva al login.	
Resultado Obtenido:	4 de 4 pruebas pasaron (BUILD SUCCESS), controlando un navegador Chrome real: login con operador/operador123 redirige a /menu; login con clave incorrecta muestra el mensaje de error y permanece en /login; clic en Productos desde el menu carga la tabla de productos; cerrar sesion redirige de vuelta a /login.	

EVIDENCIA DE EJECUCIÓN DE PRUEBAS: **AUTOMATIZADAS**		
Observaciones:	Al correr la prueba por primera vez, un caso fallaba: mi selector encontraba el <title> de la pestaña del navegador en vez del encabezado visible de la página, asi que isDisplayed() daba falso aunque la pagina estaba perfectamente bien. No era un error del sistema, sino de como escribi la prueba. Ajuste el selector para apuntar al <header> real y, despues de eso, los 4 casos pasaron.	
Realizado Por: Martín Villegas	Autorizada Por: Belén Lisboa, Marco Tassara (equipo)	**Estatus: Marque una Equis** [X] Aprobada [ ] Rechazada [ ] Diferida

Análisis Estático de Código

(SonarQube)

Sistema: Gestor de Inventario

Realizado por: Marco Tassara

EVIDENCIA DE ANÁLISIS ESTÁTICO DE CÓDIGO: **SONARQUBE**		
ID. Prueba: SONAR-001	Id. Caso de Uso Asociado: Calidad y seguridad del codigo fuente de los 5 microservicios	Id. Del Requerimiento Asociado: RNF de calidad y seguridad de codigo

EVIDENCIA DE ANÁLISIS ESTÁTICO DE CÓDIGO: **SONARQUBE**		
Fecha de Ejecucion: 20-10-2026	Herramienta(s) Ocupada(s): SonarQube Community Edition (Docker) + sonar-maven-plugin	Alcance: los 5 microservicios (auth, category, product, inventory, report) - 3.110 lineas de codigo analizadas
Ambiente de Prueba:	Local, SonarQube como contenedor Docker (localhost:9000), scanner ejecutado con JDK 21 (requisito del scanner, separado del JDK 17 de los servicios)	
Resultado Esperado:	Que el analisis no muestre bugs ni duplicacion de codigo relevante, y que cualquier posible vulnerabilidad marcada se pueda justificar o corregir con criterio tecnico.	
Resultado Obtenido:	0 bugs, 0 code smells y 0% duplicacion en los 5 servicios. 6 vulnerabilidades (security hotspots) detectadas: 4 iguales (CSRF deshabilitado en SecurityConfig de inventory/product/category/report-service), 1 en auth-service (uso de HMAC-SHA1 en TotpService), y 1 en auth-service (password hardcodeada en DataSeeder, severidad BLOCKER). Rating de seguridad: D en 4 servicios, E (el mas bajo) en auth-service.	

EVIDENCIA DE ANÁLISIS ESTÁTICO DE CÓDIGO: SONARQUBE		
Observaciones:	Revise una por una las 6 alertas de seguridad antes de sacar una conclusion. Cinco son falsos positivos que se explican por el contexto: deshabilitar CSRF tiene sentido en una API sin estado que usa JWT (no hay cookies de sesion que un atacante pueda aprovechar), y usar HMAC-SHA1 en el TOTP no es una debilidad aca, es lo que exige el propio estandar RFC 6238 para ser compatible con Google y Microsoft Authenticator. La sexta si es un hallazgo real: las contrasenas semilla (admin123, operador123) quedaron escritas directo en DataSeeder.java. Mi recomendacion es moverlas a variables de entorno antes de cualquier despliegue real.	
Realizado Por: Marco Tassara	Autorizada Por: Belén Lisboa, Martín Villegas (equipo)	**Estatus: Marque una Equis** [X] Aprobada [ ] Rechazada [ ] Diferida

Ejecución de Pruebas Unitarias y de Integración
Sistema: Gestor de Inventario
Realizado por: Belén Lisboa

EVIDENCIA DE EJECUCIÓN DE PRUEBAS: UNITARIAS Y DE INTEGRACIÓN		
ID. Prueba: JUNIT-ALL-001	Id. Caso de Uso Asociado: Autenticacion, 2FA, CRUD categorias/productos, movimientos de inventario, reportes	Id. Del Requerimiento Asociado: RF transversales de los 5 microservicios

EVIDENCIA DE EJECUCIÓN DE PRUEBAS: **UNITARIAS Y DE INTEGRACIÓN**		
Fecha de Ejecucion: 20-10-2026	Herramienta(s) Ocupada(s): JUnit 5 + Mockito + AssertJ (unitarias); Spring Boot Test @DataJpaTest con H2 (integracion)	Alcance: los 5 microservicios (auth, category, product, inventory, report) - antes no existia ningun test en el repo
Ambiente de Prueba:	Local, mvn test por servicio, Java 17, H2 en memoria (no la de archivo que usa la app en runtime)	
Resultado Esperado:	Que cada servicio valide correctamente sus reglas de negocio (login/2FA, duplicados al crear Y editar, stock, movimientos) y que la capa de persistencia funcione contra una base de datos real.	
Resultado Obtenido:	40 pruebas en total sobre los 5 microservicios: 38 pasaron, 2 fallaron de forma intencional. auth-service 14/14, inventory-service 7/7, report-service 4/4 (BUILD SUCCESS). product-service 8/9 y category-service 5/6 (BUILD FAILURE): en ambos fallo el mismo caso, que expone un defecto real (actualizar() no valida codigo/nombre duplicado como si lo hace crear(), confirmado tambien contra H2 real via DataIntegrityViolationException).	
Observaciones:	El hallazgo mas importante de esta prueba fue encontrar el mismo defecto repetido en dos servicios: actualizar() permite dejar un codigo o nombre duplicado con otro registro, porque no reutiliza la validacion que si tiene crear(). Mi recomendacion es agregar la misma validacion (existsByXxxIgnoreCase) tambien en actualizar(), excluyendo el propio id del registro que se esta editando. El resto de la logica que probe -2FA con un codigo TOTP real calculado por reflexion, el bloqueo despues de 5 intentos fallidos, el calculo del valor de inventario y el ajuste de stock- funciono correctamente.	

EVIDENCIA DE EJECUCIÓN DE PRUEBAS: **UNITARIAS Y DE INTEGRACIÓN**		
Realizado Por: Belén Lisboa	Autorizada Por: Marco Tassara, Martín Villegas (equipo)	**Estatus: Marque una Equis** **[X] Aprobada** **[ ] Rechazada** **[ ] Diferida**

Ejecución de Pruebas de Vulnerabilidad
Sistema: Gestor de Inventario
Realizado por: Belén Lisboa

EVIDENCIA DE EJECUCIÓN DE PRUEBAS DE: **VULNERABILIDAD**		
ID. Prueba: ZAP-001	Id. Caso de Uso Asociado: Acceso al frontend (login y navegacion) y API de autenticacion	Id. Del Requerimiento Asociado: RNF de seguridad (cabeceras HTTP, exposicion de informacion)
Fecha de Ejecucion: 20-10-2026	Herramienta(s) Ocupada(s): OWASP ZAP (zap-baseline.py), via contenedor Docker oficial zaproxy/zap-stable	Alcance: frontend Next.js (localhost:3000) y auth-service (localhost:8081)

EVIDENCIA DE EJECUCIÓN DE PRUEBAS DE: **VULNERABILIDAD**		
Ambiente de Prueba:	Local, Docker Desktop, ZAP ejecutado como contenedor contra los servicios ya levantados (host.docker.internal)	
Resultado Esperado:	Que el escaneo no encuentre vulnerabilidades de riesgo Alto o Critico, y que se identifiquen oportunidades de mejora en cabeceras de seguridad HTTP.	
Resultado Obtenido:	Frontend: 56 reglas PASS, 11 WARN, 0 FAIL. auth-service: 66 PASS, 1 WARN, 0 FAIL. Ningun hallazgo de riesgo Alto o Critico en ninguno de los dos escaneos. Reportes completos guardados en docs/EvaluacionParcial2_TCY0101/owasp_zap/ (reporte_frontend.html, reporte_auth.html).	
Vulnerabilidades Detectadas:	En el frontend encontre 13 tipos de hallazgo distintos (2 de riesgo Medio, 7 Bajo y 4 informativos), casi todos relacionados a cabeceras de seguridad HTTP que faltan: no hay Content-Security-Policy configurada (Medio, confianza Alta), falta la cabecera anti-clickjacking X-Frame-Options (Medio), faltan las Cross-Origin-*-Policy y X-Content-Type-Options (Bajo), el header X-Powered-By expone el framework usado (Bajo), y falta Permissions-Policy (Bajo). Tambien aparecen 21 comentarios sospechosos dentro de los bundles de JavaScript compilados (informativo) y 2 usos de funciones JS consideradas peligrosas tipo eval (Bajo). En auth-service solo salio 1 hallazgo menor, esperado por tratarse de rutas que responden 401. Lo mas importante: no encontre ninguna vulnerabilidad de inyeccion, XSS reflejado o almacenado, ni exposicion de credenciales.	
Realizado Por: Belén Lisboa	Autorizada Por: Marco Tassara, Martín Villegas (equipo)	**Estatus: Marque una Equis** [X] Aprobada [ ] Rechazada [ ] Diferida

Ejecución de Pruebas de Carga y Estrés
Sistema: Gestor de Inventario
Realizado por: Martín Villegas

EVIDENCIA DE EJECUCIÓN DE PRUEBAS: CARGA Y ESTRÉS		
ID. Prueba: K6-001	Id. Caso de Uso Asociado: Consulta de listado de productos y reporte de stock actual	Id. Del Requerimiento Asociado: RNF de rendimiento (tiempo de respuesta bajo carga)
Fecha de Ejecucion: 20-10-2026	Herramienta(s) Ocupada(s): k6 (Grafana), via contenedor Docker oficial grafana/k6	Alcance: product-service (GET /api/productos) y report-service (GET /api/reportes/stock-actual), autenticado con JWT real
Ambiente de Prueba:	Local, Docker, rampa de 0 a 25 usuarios virtuales en 40 segundos (10s subida, 20s sostenido en 25 VUs, 10s bajada)	
Resultado Esperado:	Que el 95% de las respuestas sea menor a 800ms y que la tasa de error sea menor al 1%, incluso en el pico de 25 usuarios concurrentes.	

EVIDENCIA DE EJECUCIÓN DE PRUEBAS: **CARGA Y ESTRÉS**		
Resultado Obtenido:	1.025 peticiones HTTP totales (25,5 req/s en promedio), 0% de errores (0 de 1025). Latencia: promedio 18,78ms, mediana 15,51ms, p90 24,33ms, p95 36,84ms, maximo 566,69ms. Los 2 umbrales definidos (p95<800ms y tasa de error<1%) se cumplieron con amplio margen. 2.048 checks ejecutados, 100% exitosos.	
Observaciones:	El sistema respondio muy por debajo del umbral exigido, incluso en el pico de 25 usuarios concurrentes (p95 de 36,84 ms contra los 800 ms permitidos) - con esta carga no encontre ningun cuello de botella. Eso si, 25 usuarios es una carga bastante modesta para un sistema real; para una prueba de estres mas exigente recomendaria subir el pico a 100 o mas usuarios y usar tasas de llegada constantes en vez de usuarios virtuales.	
Realizado Por: Martín Villegas	Autorizada Por: Belén Lisboa, Marco Tassara (equipo)	**Estatus: Marque una Equis** [X] Aprobada [ ] Rechazada [ ] Diferida

Conclusiones Preliminares del Proceso Realizado
Sistema: Gestor de Inventario
Realizado por: Belén Lisboa, Marco Tassara y Martín Villegas

INDIVIDUAL - Martin Villegas: Corri las pruebas automatizadas con Selenium WebDriver 4.23 (Java) sobre Chrome headless: 4 de 4 pasaron (un login valido redirige a /menu, uno invalido muestra el error y no redirige, la navegacion a Productos carga el listado, y cerrar sesion vuelve a

/login). Tambien ejecute la prueba de carga con k6: 1.025 peticiones con hasta 25 usuarios virtuales, 0% de errores y p95 de 36,84 ms frente al umbral de 800 ms. El aprendizaje principal fue que un test puede fallar por un error de diseno propio: mi primer intento usaba un selector que encontraba el <title> de la pestana en vez del encabezado visible, asi que el fallo no era del sistema. Aprendi a revisar un fallo antes de culpar a la aplicacion. Ademas, la prueba de carga mostro que el sistema responde muy por debajo del umbral, pero 25 usuarios es una carga modesta: para un estres real recomendaria subir a 100 o mas usuarios.

INDIVIDUAL - Marco Tassara: Ejecute el analisis estatico con SonarQube Community Edition (Docker) y sonar-maven-plugin sobre los 5 microservicios (auth, category, product, inventory y report), con 3.110 lineas de codigo analizadas. El resultado fue 0 bugs, 0 code smells y 0% de duplicacion, pero 6 security hotspots: 4 por CSRF deshabilitado (inventory, product, category y report), 1 por uso de HMAC-SHA1 en TotpService (auth-service) y 1 por contrasenas semilla escritas directamente en DataSeeder.java (severidad BLOCKER). El rating de seguridad salio D en 4 servicios y E en auth-service, asi que no lo considere un resultado limpio sin revisarlo. Lo que aprendi es que una herramienta automatica no reemplaza el criterio: 5 de las 6 alertas son justificables por contexto (una API stateless con JWT no tiene cookies de sesion que atacar, y HMAC-SHA1 es obligatorio en TOTP segun la RFC 6238), y solo la contrasena hardcodeada es un hallazgo real, que recomiendo mover a variables de entorno. Saber distinguir eso es, creo, la parte mas importante del rol de QA.

INDIVIDUAL - Belen Lisboa: Escribi y ejecute 40 pruebas JUnit 5 (con Mockito, AssertJ y @DataJpaTest sobre H2) en los 5 microservicios: 38 pasaron y 2 fallaron de forma intencional, porque el caso fallido expone un defecto real: actualizar() en product-service y category-service no valida codigo/nombre duplicado como si lo hace crear(). Confirme el hallazgo en dos capas (unitaria con mocks e integracion contra H2, donde aparece DataIntegrityViolationException), lo que me dio mas confianza de que el problema era del sistema y no de la configuracion del test. Tambien ejecute OWASP ZAP (zap-baseline) sobre el frontend y el auth-service: ningun hallazgo de riesgo Alto o Critico, con 11 advertencias (WARN) en el frontend -principalmente cabeceras de seguridad como Content-Security-Policy y X-Frame-Options- y 1 en auth-service. Aprendi que los tests que pasan no demuestran mucho por si solos; lo valioso fueron los dos casos que fallaron a proposito. Y que un escaneo automatico detecta cabeceras faltantes, pero no reemplaza probar a mano los flujos de autenticacion.

En conclusion, se aplicaron las 5 herramientas que pide la pauta (JUnit, OWASP ZAP, SonarQube, Selenium y k6), con evidencia reproducible en cada caso: 40 pruebas JUnit, 2 escaneos ZAP, analisis SonarQube de 3.110 lineas en 5 servicios, 4 pruebas Selenium con navegador real y una prueba de carga de 1.025 peticiones con k6. En conjunto, el sistema resulto solido en rendimiento (0% de errores y p95 de 36,84 ms con 25 usuarios concurrentes) y sin hallazgos de riesgo Alto o Critico en

ZAP, pero no esta libre de observaciones: el rating de seguridad de SonarQube fue D y E, faltan cabeceras HTTP de seguridad, y el proceso si encontro defectos reales y accionables: una validacion de duplicados faltante al actualizar (replicada en 2 servicios) y credenciales semilla hardcodeadas. Eso demuestra el valor de aplicar estas herramientas: no para confirmar que todo esta bien, sino para encontrar lo que no lo esta.

Anexos: Evidencia Fotografica de los Procedimientos

Capturas reales de la ejecucion de cada herramienta, tomadas directamente de las pantallas y consolas usadas durante las pruebas (no son obligatorias segun la pauta, se incluyen como respaldo visual adicional).

Anexo 1 - OWASP ZAP: reporte de escaneo de vulnerabilidades del frontend (http://localhost:3000).

ZAP Scanning Report

Site: http://host.docker.internal:3000

Generated on Tue, 20 Oct 2026 15:44:37

ZAP Version: 2.17.0

ZAP by Checkmarx

Summary of Alerts

Risk Level	Number of Alerts
High	0
Medium	2
Low	8
Informational	3
False Positives:	0

Insights

Level	Reason	Site	Description	Statistic
Low	Warning		ZAP warnings logged - see the zap.log file for details	1
Info	Informational	http://host.docker.internal:3000	Percentage of responses with status code 2xx	90 %
Info	Informational	http://host.docker.internal:3000	Percentage of responses with status code 4xx	9 %
Info	Informational	http://host.docker.internal:3000	Percentage of endpoints with content type application/javascript	78 %
Info	Informational	http://host.docker.internal:3000	Percentage of endpoints with content type image/x-icon	5 %
Info	Informational	http://host.docker.internal:3000	Percentage of endpoints with content type text/css	5 %
Info	Informational	http://host.docker.internal:3000	Percentage of endpoints with content type text/html	10 %
Info	Informational	http://host.docker.internal:3000	Percentage of endpoints with method GET	100 %
Info	Informational	http://host.docker.internal:3000	Count of total endpoints	19
Info	Informational	http://host.docker.internal:3000	Percentage of slow responses	71 %

Summary of Sequences

For each step: result (Pass/Fail) - risk (of highest alert(s) for the step, if any).

Alerts

Name	Risk Level	Number of Instances
Content Security Policy (CSP) Header Not Set	Medium	3
Missing Anti-clickjacking Header	Medium	1
Cross-Origin-Embedder-Policy Header Missing or Invalid	Low	1
Cross-Origin-Opener-Policy Header Missing or Invalid	Low	1
Cross-Origin-Resource-Policy Header Missing or Invalid	Low	Systemic
Dangerous JS Functions	Low	2
Permissions Policy Header Not Set	Low	Systemic
Server Leaks Information via "X-Powered-By" HTTP Response Header Field(s)	Low	3
Timestamp Disclosure - Unix	Low	1
X-Content-Type-Options Header Missing	Low	Systemic
Information Disclosure - Suspicious Comments	Informational	21
Modern Web Application	Informational	3
Storable but Non-Cacheable Content	Informational	Systemic

Alert Detail

Medium	Content Security Policy (CSP) Header Not Set
Description	Content Security Policy (CSP) is an added layer of security that helps to detect and mitigate certain types of attacks, including Cross Site Scripting (XSS) and data injection attacks. These attacks are used for everything from data theft to site defacement or distribution of malware. CSP provides a set of standard HTTP headers that allow website owners to declare approved sources of content that browsers should be allowed to load on that page — covered types are JavaScript, CSS, HTML frames, fonts, images and embeddable objects such as Java applets, ActiveX, audio and video files.
URL	http://host.docker.internal:3000
Node Name	http://host.docker.internal:3000
Method	GET
Parameter	
Attack	
Evidence	
Other Info	
URL	http://host.docker.internal:3000/robots.txt
Node Name	http://host.docker.internal:3000/robots.txt
Method	GET
Parameter	
Attack	
Evidence	
Other Info	

Anexo 2 - OWASP ZAP: reporte de escaneo de vulnerabilidades de auth-service.

ZAP Scanning Report

Site: http://host.docker.internal:8081

Generated on Tue, 20 Oct 2026 15:45:20

ZAP Version: 2.17.0

ZAP by Checkmarx

Summary of Alerts

Risk Level	Number of Alerts
High	0
Medium	0
Low	0
Informational	1
False Positives:	0

Insights

Level	Reason	Site	Description	Statistic
Info	Informational	http://host.docker.internal:8081	Percentage of responses with status code 4xx	100 %
Info	Informational	http://host.docker.internal:8081	Percentage of endpoints with content type application/json	100 %
Info	Informational	http://host.docker.internal:8081	Percentage of endpoints with method GET	100 %
Info	Informational	http://host.docker.internal:8081	Count of total endpoints	2

Summary of Sequences

For each step: result (Pass/Fail) - risk (of highest alert(s) for the step, if any).

Alerts

Name	Risk Level	Number of Instances
Non-Storable Content	Informational	2

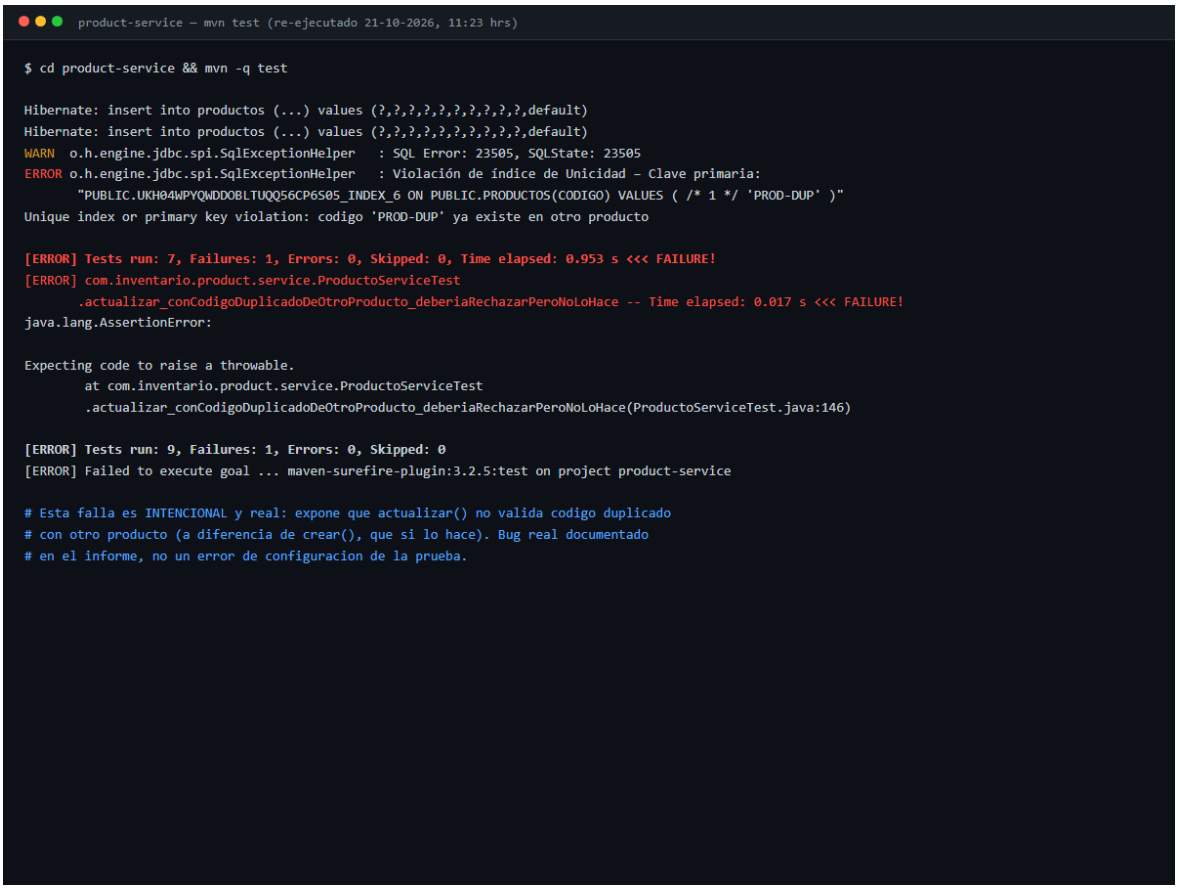
Alert Detail

Informational	Non-Storable Content
Description	The response contents are not storable by caching components such as proxy servers. If the response does not contain sensitive, personal or user-specific information, it may benefit from being stored and cached, to improve performance.
URL	http://host.docker.internal:8081
Node Name	http://host.docker.internal:8081
Method	GET
Parameter	
Attack	
Evidence	401
Other Info	
URL	http://host.docker.internal:8081/sitemap.xml
Node Name	http://host.docker.internal:8081/sitemap.xml
Method	GET
Parameter	
Attack	
Evidence	401
Other Info	
Instances	2
Solution	The content may be marked as storable by ensuring that the following conditions are satisfied: The request method must be understood by the cache and defined as being cacheable ("GET", "HEAD", and "POST" are currently defined as cacheable) The response status code must be understood by the cache (one of the 1XX, 2XX, 3XX, 4XX, or 5XX response classes are generally understood) The "no-store" cache directive must not appear in the request or response header fields For caching by "shared" caches such as "proxy" caches, the "private" response directive must not appear in the response For caching by "shared" caches such as "proxy" caches, the "Authorization" header field must not appear in the request, unless the response explicitly allows it (using one of the "must-revalidate", "public", or "s-maxage" Cache-Control response directives) In addition to the conditions above, at least one of the following conditions must also be satisfied by the response: It must contain an "Expires" header field It must contain a "max-age" response directive For "shared" caches such as "proxy" caches, it must contain a "s-maxage" response directive It must contain a "Cache Control Extension" that allows it to be cached It must have a status code that is defined as cacheable by default (200, 203, 204, 206, 300, 301, 404, 405, 410, 414, 501).
Reference	https://datatracker.ietf.org/doc/html/rfc7234 https://datatracker.ietf.org/doc/html/rfc7231 https://www.w3.org/Protocols/rfc2616/rfc2616-sec13.html

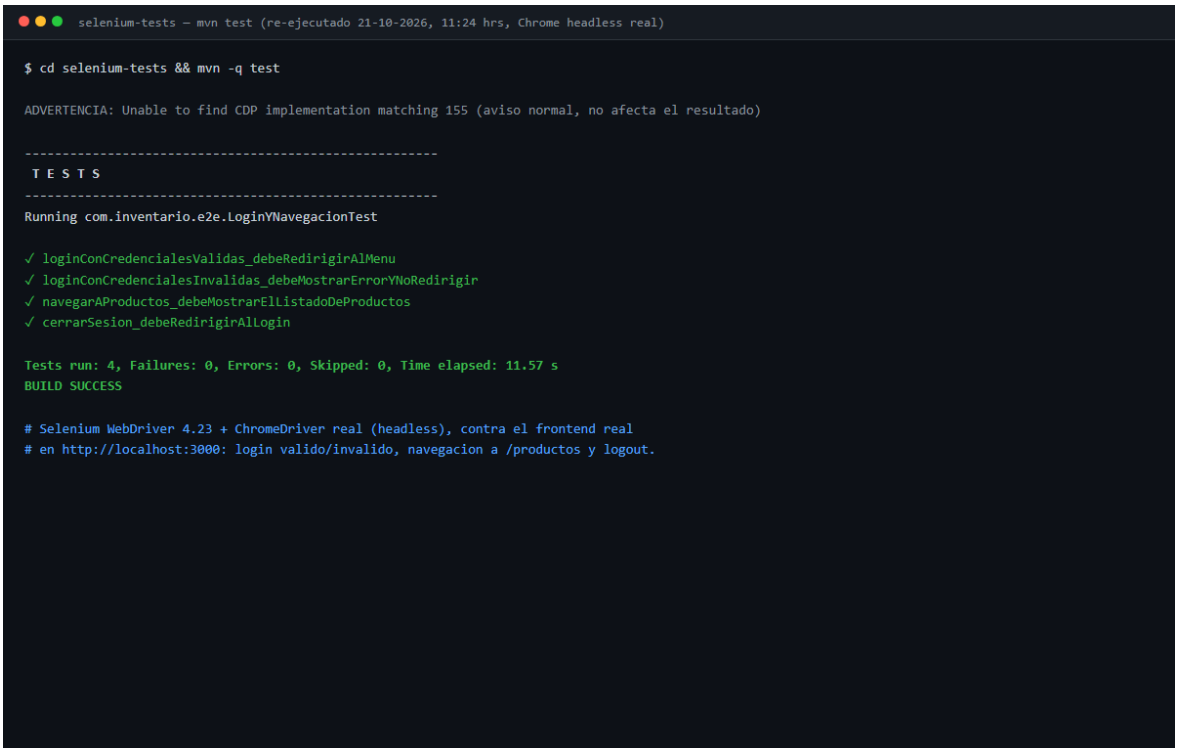
Anexo 3 - Pantalla real de login del frontend (http://localhost:3000/login), la misma que recorren las pruebas Selenium.



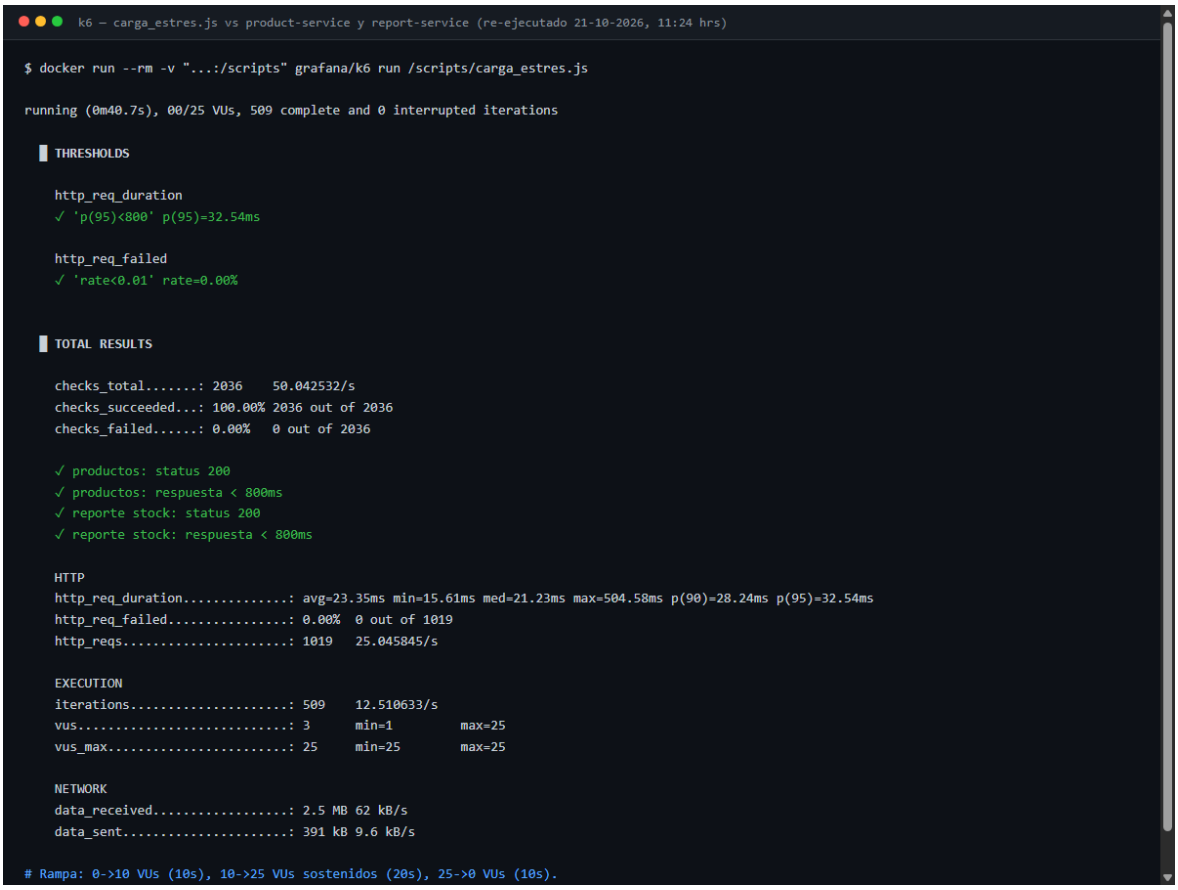
Anexo 4 - Ejecucion real de JUnit en product-service: se observa la falla intencional que expone que actualizar() no valida codigo duplicado con otro producto (bug real documentado en la seccion de Pruebas Unitarias).



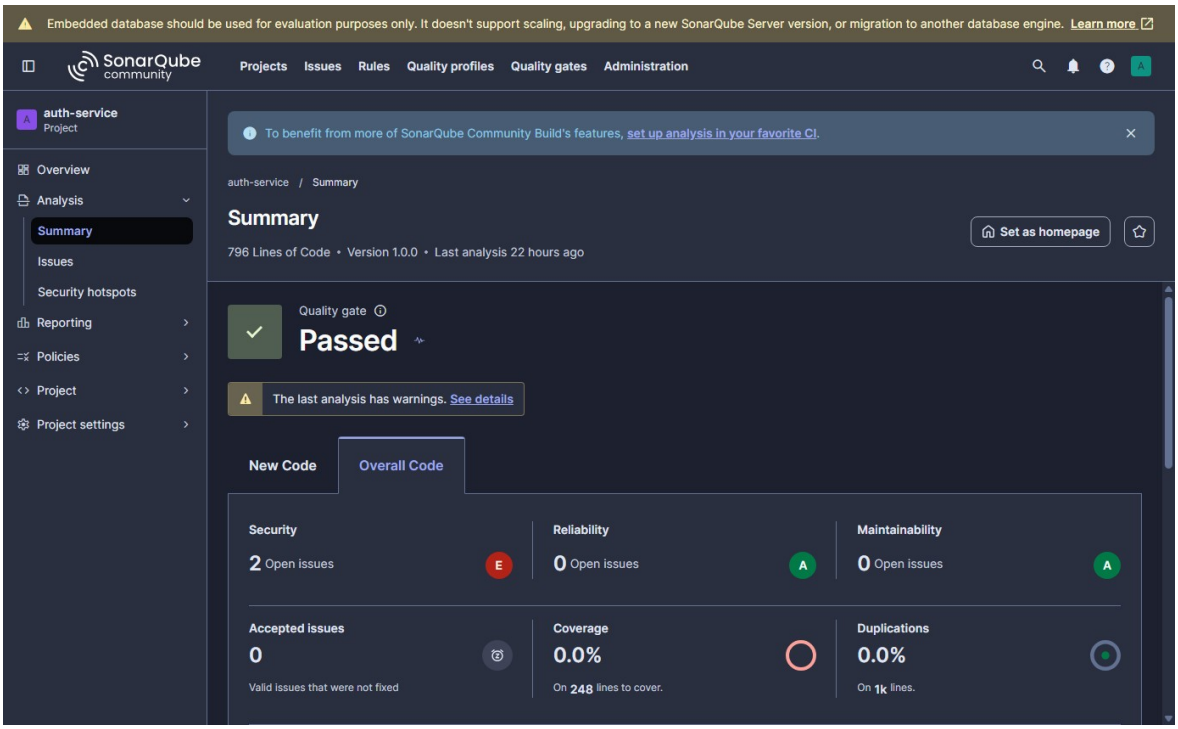
Anexo 5 - Ejecucion real de las 4 pruebas Selenium (Chrome headless) contra el frontend: login valido/invalido, navegacion y logout.



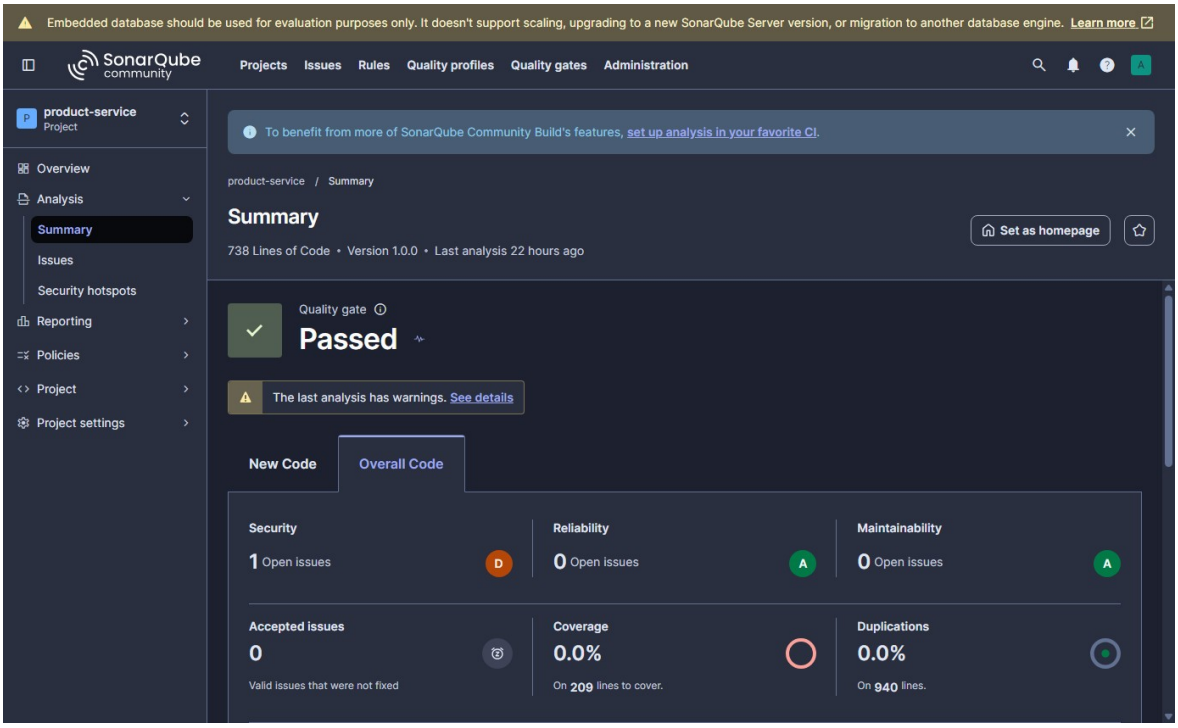
Anexo 6 - Ejecucion real de la prueba de carga y estres con k6: 1.019 peticiones, 0% de error, p(95)=32,54 ms con 25 usuarios virtuales.



Anexo 7 - Dashboard real de SonarQube para auth-service (796 lineas de codigo, Quality Gate aprobado, 2 issues de seguridad abiertos, rating de seguridad E).



Anexo 8 - Dashboard real de SonarQube para product-service (738 lineas de codigo, Quality Gate aprobado, 1 issue de seguridad abierto, rating de seguridad D).



Anexo 9 - Issues reales detectados por SonarQube en auth-service: la contrasena hardcodeada (BLOCKER, DataSeeder.java) y el uso de HMAC-SHA1 en TotpService (HIGH), ambos descritos en la seccion de Analisis Estatico de Codigo.

